

Strong at the Code, Weak at the Body:

A First-Iteration Study of LLMs in Robotics Development

CS 7980 Capstone, Northeastern University Vancouver
GuideMate Project · Work-in-Progress Paper, Iteration 1

Abstract

This paper tests a single working thesis: large language models are strong at the software-engineering side of robotics—writing, repairing, and debugging ROS/ROS2 code through closed-loop iteration—but weak as the runtime embodied brain responsible for low-level sensorimotor control, spatial reasoning, multi-robot coordination, and runtime error recovery. We approach the question from two evidence sources held to one standard: every factual claim must trace either to a literature source whose URL was adversarially fetched and confirmed, or to a citation in our own conversation corpus (file, date, turn) backed by a raw transcript. On the literature side we synthesize 78 verified findings across general software repair, robotics code and debugging, embodied control, named agentic tools, and benchmark reliability. On the primary-evidence side we analyze 30 of our own Claude Code sessions (2,153 user turns, 4,192 assistant turns, 1,697 tool calls) from a real TurtleBot 4 capstone project. The headline finding is corroboration: across 28 non-empty sessions the agent reached or partially reached its goal in nearly every case, every session classified as software engineering supported the thesis, and every place the work crossed into the physical runtime—verifying a GUI, confirming motion, docking, surviving a brownout—is where the agent could not close the loop. The split is not a deficiency of one model on one day; it is a structural boundary between reasoning about code that controls a robot and being the controller.

1. Research Question and Thesis

The research question is: *Can we use Large Language Models in Robotics Development?* The working thesis being tested is more precise than yes or no. It claims an asymmetry. LLMs are expected to be strong at the software-engineering side of robotics—authoring ROS/ROS2 nodes, configuration, and launch files, and repairing and debugging them through closed-loop write-run-read-error-fix iteration. They are expected to be weak as the runtime embodied brain: low-level sensorimotor control, spatial reasoning, multi-agent coordination at runtime, and recovery from physical execution errors.

The goal of iteration 1 is to test whether that asymmetry holds against two independent kinds of evidence: published literature and our own working sessions on a real robot. If both agree, the thesis is corroborated and the boundary becomes a design constraint we can build around. If they diverge, the divergence itself is the finding.

2. Methodology

2.1 Two Evidence Streams, One Standard

Every claim in this paper is held to one rule: it must be traceable. A claim about the literature must cite a source whose URL appears in our verified findings file. A claim about our own work must cite a session as (file, date, turn) and refer to the raw transcript under the sessions directory. Corpus statistics use only measured numbers from the INDEX and stats files. Where we were tempted to add outside knowledge that was not in these inputs, we omitted it.

2.2 The Conversation Corpus: Collection, Redaction, Deterministic Digest

The primary evidence is our own usage. Sessions were collected by a configurable, tested collector built and exercised inside the project itself. Two sessions in the corpus document that very tooling being built: a reusable cross-laptop tool to collect Claude Code transcripts, strip internal metadata, redact secrets, and store the result in git (2026-06-22_610e67b0.jsonl, turn 1), and a later run of the resulting collect-claude-conversations skill across all conversation folders on the device (2026-06-22_6014c160.jsonl, turn 1). Redaction is not an afterthought: in one build, a proactive secret scan found a live AWS access key ID and a real password, and the agent refused to push until they were removed, with an independent grep confirming zero remaining instances (2026-06-22_610e67b0.jsonl, turns 81, 148). From the redacted JSONL we produced a deterministic digest per session and a stats.json of measured counts. Corpus totals: 30 sessions analyzed, 2,153 user turns, 4,192 assistant turns, 1,697 tool calls, 178 tasks recorded done and 46 failed or abandoned.

2.3 Agent Analysis of Each Conversation

Each digest was analyzed and recorded in _analyses.json with a fixed schema: motive, outcome (reached, partial, not-reached), thesis category (software-engineering, mixed, non-technical, embodied-hardware), automation level, measured turn and tool counts, a thesis-alignment paragraph, and a list of key_evidence items each carrying a (file, date, turn) citation back to the raw transcript. This lets every interpretive statement in Section 4 be checked against a specific point in a specific session.

2.4 Literature: Research, Then Adversarial URL Verification

The literature was rebuilt under verification. Finder agents searched per cluster. Every candidate finding was sent to two independent skeptic agents that fetched the cited URL and judged whether the page actually supported the claim. A finding is kept only if at least one skeptic confirmed support and none refuted it. Of 91 candidates, 78 were kept and 13 dropped—an 86 percent keep rate. The drops are instructive: a claimed RoboCoach result was refuted because the source reports a 26.5 percent margin and no such figures; an EmbodiedBench framing was downgraded to cannot-verify because the abstract attributes 28.9 percent to GPT-4o's overall average, not low-level manipulation; a Silo-Bench claim was caught when deterministic PDF extraction showed earlier fetch summaries had hallucinated the supporting text. The numbers we cite below are the ones that survived.

2.5 Acknowledged Gaps

A completeness critic flagged where the thesis is least supported by direct evidence: multi-robot coordination as a runtime phenomenon is thin; the perception-versus-reasoning confound is real because several strong embodied results assume privileged state; error recovery as a distinct runtime weakness is scattered rather than isolated; and sim-to-real validation depth is uneven. These gaps are carried forward honestly into Section 6.

3. Literature Review

3.1 General Software Engineering: Generation, Repair, Debugging

The trajectory on software repair is steep. When SWE-bench launched (ICLR 2024), the best frontier model resolved only 1.96 percent of 2,294 real GitHub issue tasks. Multi-agent debuggers then posted strong gains. FixAgent/UniDebugger set a new state of the art on Defects4J by correctly fixing 197 bugs, beating prior SoTA ChatRepair by 25.48 percent and fixing all bugs in QuixBugs; on repository-level Defects4J it fixes 1.25× to 2.56× more bugs than prior methods without fault localization. RepairAgent's

closed-loop write-run-validate iteration is cheap, averaging 270,000 tokens per bug (about 14 cents at GPT-3.5 pricing). RGD, a three-agent (Guide, Debug, Feedback) debugger, reached state of the art with 9.8 percent improvement on HumanEval and 16.2 percent on MBPP; ablations show the decomposition itself matters. CodeSim reached pass@1 of 95.1 percent on HumanEval and 90.7 percent on MBPP.

The mechanism behind these gains is closed-loop iteration, and the residual weakness is semantic, not syntactic. Failure analysis of agent-generated patches attributes 47 percent to logic errors and 35 percent to incomplete fixes, while syntactic errors account for only about 14 percent. Multi-agent frameworks also carry costs: ChatDev reports high executability (0.88) but only moderate completeness (0.56) and its authors warn the systems suit prototypes rather than complex real-world applications; MetaGPT reaches 85.9 percent Pass@1 on HumanEval but consumes 31,255 tokens per task versus ChatDev's 19,292.

3.2 Robotics-Specific Code Generation and Debugging

The strength carries into robotics code with the same closed-loop signature. OperateLLM integrates ROS2 tools into an LLM via ReAct and generates fundamental ROS2 components like Nodes and Publishers. LLM-based agents can recover hierarchical architecture models directly from source code and launch files across three repositories, including an industrial-scale subset. A closed-loop generate-simulate-evaluate framework for drone/UAV code raised task success from 33.3 to 55 percent (direct generation) up to 85 to 93.3 percent with iterative error-fix correction in AirSim and PX4-Gazebo—direct evidence that write-run-read-error-fix loops sharply improve real robot control code. RoboRepair traces execution to failure and runs an LLM-produced recovery program over eleven error-condition tasks.

The most thesis-relevant robotics result is a detection-versus-root-cause gap. ROS Help Desk achieved 100 percent accuracy at detecting which injected error type occurred but only 18 percent at identifying the specific injected fault, averaging 68 percent across eight debugging criteria while its ROSA baseline reached only 29 percent detection. This is the software-engineering strength and its ceiling in one experiment: near-perfect surface detection, weak deep diagnosis. RoboEval makes the same point on the generation side: even GPT-4 reached pass@1 ≥ 0.75 on only 67.5 percent of prompts, with hallucinated invalid locations or objects dominating execution errors. Closed-loop policy development can be very strong: RoboCoach-Bench shows the agentic loop surpassing human-engineered baselines by 26.5 percent under binary task-completion criteria, with optimized policies improving real-robot success from 0.51 to 0.68 over four transfer tasks. FAEA applies an unmodified Claude Agent SDK directly to embodied manipulation, reaching 84.9 percent (LIBERO), 85.7 percent (ManiSkill3), and 96 percent (MetaWorld)—but with privileged state access and zero demonstrations.

3.3 Embodied Control: Where LLMs Remain Weak

When the model is asked to be the runtime brain rather than to write code, the picture inverts. On EmbodiedBench, GPT-4o scores 52 percent (ALFRED) and 36 percent (Habitat) on high-level spatial awareness but only 25.0 percent on EB-Manipulation spatial awareness and 19.4 percent on visual appearance. On Butter-Bench (real robot, LLM as high-level controller), the best LLM (Gemini 2.5 Pro) scores 40 percent versus a mean human score of 95 percent, with Claude Opus 4.1 at 37 percent and Llama 4 Maverick at 7 percent. On PARTNR (100,000 multi-agent collaboration tasks), humans solve 93 percent but SoTA LLMs complete only 30 percent under non-privileged conditions. On RoboBench, all models degrade by about 30 percent on implicit instructions. On VLABench, SoTA VLAs perform near zero on composite tasks (Octo 0.00 percent, OpenVLA 2.66 percent, RDT-1B 3.34 percent). On BLINK, humans average 95.70 percent on core spatial perception while GPT-4V reaches only 51.26 percent.

Three runtime failure modes recur. Coordination: on PARTNR, decentralized two-agent LLM coordination is worse than a single agent, taking 31 percent more simulation steps and a 300 percent increase in extraneous effort; DPBench shows agents handle sequential but not simultaneous coordination, with N=5 deadlock from 25.0 percent (GPT-5.2) to 90.0 percent (Gemini 2.5 Flash). Error recovery: ROSClaw documents replan loops where the model repeatedly reissues a blocked command without adapting parameters in 19 percent of cases (31 percent for Llama 4, 4 percent for Claude), with no automatic loop-breaker; a Conditional Multi-Stage Failure Recovery baseline succeeds on only 24.86 percent of seen tasks. Perception as a confound: moving from oracle to learned perception on object-search drops success by about 25.8 points; on LIBERO-Occ, privileged visual access yields 22.1 to 45.5 point gains under occlusion. FAEA explicitly relies on privileged ground-truth state via `get_obs` rather than raw RGB.

3.4 Named Agentic Tools and Benchmark Reliability

Named-tool evidence is mostly grey literature and mostly unvalidated on hardware. NASA-JPL's ROSA wraps standard ROS/ROS2 utilities into tool-enabled functions but reports no low-level-control evaluation metrics and explicitly limits the agent to an operator interface that cannot autonomously interpret a command like "move toward the rock" without structured waypoint input. AgenticROS connects ROS2 to commercial agentic tools but is undocumented and unvalidated. The ROSBag MCP Server found a large divide in tool-calling capability across eight models, with Claude Sonnet 4 and Kimi K2 clearly superior. The strong software numbers must also be read against contamination and weak-test effects. On the contamination-resistant SWE-Bench Pro, the best frontier models score only about 23 percent on the public set versus over 70 percent on SWE-bench Verified—a roughly 47-point gap. OpenAI publicly stopped reporting Verified, having audited 138 tasks and found 59.4 percent contained material test-design or problem-description flaws. The SWE-Bench Illusion shows memorization at work, with Claude 4 Opus reproducing benchmark code verbatim in 31.6 percent of instances. The net of Section 3 is the split the thesis predicts: strong, closed-loop competence on software and robotics code, with a hard wall at embodied runtime control, and headline software numbers that overstate production reliability.

4. Our Conversations as Primary Evidence

This is the novel contribution. We corroborate the literature thesis not only from published benchmarks but from 30 of our own working sessions on a real TurtleBot 4 guide robot. Because we authored the project, we know exactly what the agent was asked to do and what the physical robot did in response.

4.1 The Measured Distribution

Across 30 sessions there were 2,153 user turns, 4,192 assistant turns, and 1,697 tool calls, with 178 tasks recorded done against 46 failed or abandoned. The outcome distribution is reached 16, partial 12, not-reached 2. The two not-reached sessions are both empty housekeeping stubs (a `/model` command and a `/clear` command) with zero assistant turns. Removing those, every substantive session reached or partially reached its goal. The thesis-category distribution is the core result: software-engineering 21, mixed 4, non-technical 4, embodied-hardware 1. Automation level is high 22, medium 6, not-determinable 2. In words: 21 of 30 sessions are squarely software-engineering work on a robotics codebase, and 22 of 30 ran at high automation. The one session categorized as embodied-hardware is the live docking attempt discussed below.

4.2 Where the Agent Was Strong: Code, Config, Debugging

The software-engineering sessions are uniformly strong and uniformly support the thesis. The heaviest robotics-code session—autonomous mapping development for robot 468 (457 user turns, 1,025 assistant turns, 379 tool calls)—shows the agent build a custom BFS frontier explorer and validate it with a read-only dry run that found 19 frontier clusters with no motion (2026-06-19_69022496.jsonl, turn 167), diagnose and fix a namespaced-TF remap bug without which the explorer would never receive a pose (turn 158), correctly diagnose the Create 3 hazard stream as event-only by showing 96 messages in five seconds on odom while hazard stayed silent (turn 349), and find a camera_info 1280×720 versus depth 640×480 mismatch that had skewed the depth wedge to about minus 89 degrees (turn 1073). This is closed-loop write-run-read-error-fix on real ROS2 code—exactly the capability the literature credits.

The same competence appears across the corpus. The C++ porting session faithfully ported three nontrivial Python nodes to rclcpp including numpy float32 parity, caught its own bugs via multi-agent adversarial review, and benchmarked real speedups of roughly 14× for fusion, 17× for the explorer, and 10× for glass_guard (2026-06-20_29aff2bd.md, turn 1216). The deepest debugging session ran a single-variable experiment proving the OAK-D camera wedge root cause was usbfs_memory_mb=16, not power: usbfs=16 aborts in about 5 seconds with X_LINK_ERROR while usbfs=256 runs 199 seconds with zero disconnects (2026-06-20_7833e513.jsonl, turn 101), explicitly overturning the project's prior documented root cause. Network and OS plumbing was equally solid: enterprise WiFi onboarding fixed regdom, the 802.1X profile, a stale discovery address, a corrupted FastDDS discovery database, and NTP, ending with end-to-end ROS2 over NUwave confirmed (2026-06-12_284d799e.jsonl, turn 359). Documentation-audit and research subagents (a0, a1, a5–a9, ac–af) all reached their goals at high automation, statically verifying docs against on-disk code and pivoting around 404s to working endpoints. These are the desk-bound, over-the-text tasks the thesis predicts LLMs do well, and the corpus shows it doing them well.

4.3 Where the Agent Hit the Embodiment Boundary

The thesis predicts the agent weakens precisely when the task crosses into the physical runtime. The corpus shows this repeatedly. It could not verify what it could not sense. In the simulation bring-up session, the agent did strong systems work (recovered a hung WSL, installed ROS2 Humble, root-caused robot-not-spawning as a timing race) but could not observe the GUI or whether the robot moved, depending on user screenshots for ground truth (2026-06-03_d29ce515.jsonl, turns 337, 369, 490, 543). In the presentation session, the agent claimed two final fixes "verified" despite 0-character image reads (2026-06-15_40f8e5f3.jsonl, turn 135). The robot-mapping session leaned on the human for physical ground truth and had to be corrected on spatial reasoning about the camera mount (2026-06-19_69022496.jsonl, turn 456).

It drove into things and could not recover at runtime. The first autonomous run drove the robot into the glass wall and required killing all of Nav2 to stop it (turn 257); even the glass-aware re-run still tapped the glass repeatedly because the reactive mark-and-blacklist did not block the path fast enough (turn 428). In the clearest single embodied-hardware case—a live docking attempt—the agent fluently used the ROS2 CLI to read battery as 23 percent discharging and to dispatch the Dock action with the correct message type, but the action finished ABORTED with is_docked false, and the agent gave an honest but non-recovering report, attempting no dock_status diagnosis or retry (2026-06-20_6fb45153.md, turns 27, 28). This is the literature's bounded-recovery failure mode in our own logs.

The hardest wall is physical power delivery. The long power-and-port session could not get the perception/SLAM/Nav2 brain to run on the robot because the Pi hit its compute wall and Nav2 lifecycle never activated; the robot never moved autonomously (2026-06-20_29aff2bd.md, turns 561, 575). The camera investigation characterized but could not fix a residual heavy-RGBD-plus-USB3 brownout, acknowledging the true fix is power-delivery hardware (2026-06-20_7833e513.jsonl, turn 470). Off-robot, confident-but-wrong is a real risk: a pure research subagent produced a polished, well-cited verdict that the camera disconnect was an on-board 5V brownout, a conclusion later contradicted by on-device testing in favor of the usbfs kernel fix (2026-06-20_agent-a6.jsonl, turn 58). Away from the running robot the agent is fluent and plausible; the robot is what falsifies it.

4.4 Reading the Distribution Against the Thesis

The distribution corroborates the thesis. The work the team actually delegated to the agent is overwhelmingly software engineering (21 of 30 sessions); the agent ran it at high automation (22 of 30) and reached or partially reached almost every non-empty goal (26 of 28). Every session classified as software-engineering or mixed that touched robot code was assessed as evidence for the thesis on its software-engineering half. The places it stalled are not random; they cluster exactly at the embodiment boundary the thesis names: verifying a rendered GUI, confirming motion, docking, and surviving hardware power limits. The corpus does not merely fail to contradict the thesis; its strong and weak points fall on the predicted sides of the line.

5. Discussion

Our logs agree with the literature thesis, and they agree on mechanism, not just outcome. On the strong side, the literature attributes robotics-code gains to closed-loop write-run-read-error-fix iteration rather than single-shot generation, and that is exactly the loop our sessions ran: the usbfs single-variable experiment, the namespaced-TF fix verified by re-run, and the multi-agent adversarial C++ verification are local instances of the same machinery. The literature's detection-versus-root-cause gap (100 percent detection, 18 percent root cause in ROS Help Desk) has a striking parallel: our agent reliably detected and explained the camera wedge but its off-robot root-cause verdict was wrong until on-device experiment corrected it.

On the weak side, the literature's three named runtime failures all surface in our corpus. Error recovery: ROSClaw's bounded-recovery failure matches our aborted dock with no retry and the glass-wall collision that needed a manual kill. Spatial reasoning: BLINK and the spatial benchmarks match the user having to correct the agent on camera-mount geometry. The "agent cannot verify the physical world itself" pattern, repeated across our GUI and motion cases, is the practical face of the perception confound the literature isolates with privileged-state ablations.

Two honest points of divergence remain. First, our corpus contains almost no multi-robot coordination evidence: all substantive work is on a single robot (468), so the coordination pillar of the thesis is tested by the literature (PARTNR, DPBench) and not by us—the same gap the completeness critic flagged. Second, our embodiment failures are mostly about verification and physical limits rather than an LLM directly emitting low-level motor commands. We deliberately did not put an LLM in the inner control loop; our C++ explorer port is goal-level only and never publishes cmd_vel. So our evidence supports the weak-embodied-brain thesis at the integration and verification boundary strongly, and supports the narrow "LLM as low-level sensorimotor controller fails" claim only indirectly—consistent with how the

strongest embodied results in the literature also keep the LLM out of the tightest loop and fence sensorimotor execution to classical or learned controllers.

6. Limitations and Future Work

This is a first iteration, and several limitations bound its strength. Self-study bias: we analyzed our own sessions on our own project. The same team set the tasks, ran the agent, and judged the outcomes. The thesis-alignment labels are interpretive, even though every one is tied to a (file, date, turn) citation, and an independent re-analysis could shift the mixed and non-technical labels. Redaction gaps: the corpus was redacted by an automated secret scanner that did catch real credentials (2026-06-22_610e67b0.jsonl, turn 81), but some sessions nevertheless record secrets having been exposed in chat before redaction (2026-06-12_284d799e.jsonl, turns 35, 397, 400); redaction is best-effort and absence of a flagged leak is not proof of none. Simulation versus real: our strongest agent wins are split between real-robot work and off-robot code, research, and docs, while much literature strength is simulation-only or assumes privileged state (FAEA, RoboCoach), and our clearest embodiment failures are real-hardware (docking, brownout); the sim-to-real degradation question is not resolved here. Single-team, single-robot sample: thirty sessions from one capstone team on one robot model is a narrow base, and multi-robot coordination, the most theoretically interesting runtime weakness, is essentially untested in our data.

Future iterations should: run a controlled ROS2 bug-fix experiment with held-out tasks to directly measure the SWE-for-robotics ceiling; replicate a draft-debug-improve policy loop in simulation with privileged state then degrade to RGB to localize the perception confound; and deliberately stage a two-robot coordination task—we have a second unit, robot 436—to gather the multi-robot runtime evidence our current corpus lacks. The overall design takeaway already emerging is a structural one: treat LLMs as the software-engineering layer of a robot system and not as its runtime brain. Closed-loop coding agents earn delegation; the inner control loop, the verification of physical state, and recovery from runtime hardware failures still belong to deterministic code and to humans.

References

Web sources used (each verified by URL fetch in Research/literature/findings.md):

- SWE-bench: Can Language Models Resolve Real-World GitHub Issues? <https://openreview.net/pdf?id=VTF8yNQM66>
- SWE-ABS: Adversarial Benchmark Strengthening. <https://arxiv.org/pdf/2603.00520>
- UniDebugger (FixAgent), arXiv. <https://arxiv.org/abs/2404.17153>
- UniDebugger (FixAgent), EMNLP 2025. <https://aclanthology.org/2025.emnlp-main.921/>
- RepairAgent. <https://arxiv.org/abs/2403.17134>
- RGD: Multi-LLM Based Agent Debugger. <https://arxiv.org/abs/2410.01242>
- RGD (HTML version). <https://arxiv.org/html/2410.01242v2>
- CodeSim. <https://arxiv.org/abs/2502.05664>
- ChatDev. <https://arxiv.org/html/2307.07924v5>
- MetaGPT. <https://arxiv.org/html/2308.00352v6>
- ROS2 architecture recovery with LLM-based agents. <https://arxiv.org/abs/2602.18644>
- ROS2 architecture recovery (HTML). <https://arxiv.org/html/2602.18644v1>
- OperateLLM. <https://ieeexplore.ieee.org/document/10730448/>
- robotics-agent-skills. <https://github.com/arpitg1304/robotics-agent-skills>
- ROS Help Desk. <https://arxiv.org/abs/2507.07846>
- ROS Help Desk (HTML). <https://arxiv.org/html/2507.07846v1>
- RoboCoach / From Digital to Physical. <https://arxiv.org/abs/2601.21570>
- RoboCoach (HTML). <https://arxiv.org/html/2601.21570>
- FAEA: Demonstration-Free Robotic Control via LLM Agents. <https://arxiv.org/abs/2601.20334>
- FAEA author summary (privileged-state caveat). <https://medium.com/@brianytsui/claude-code-as-embodied-agent-to-control-robots-85-96-success-in-sim-with-zero-demonstration-455a9044d353>
- ROSA (NASA-JPL). <https://arxiv.org/abs/2410.06472>
- CodeBotler / RoboEval. <https://arxiv.org/abs/2311.11183>
- RoboEval (HTML). <https://arxiv.org/html/2311.11183v3>
- LLM-Driven Corrective Robot Operation Code Generation (UAV). <https://arxiv.org/html/2512.02002v1>
- RoboRepair. <https://arxiv.org/abs/2410.18893>
- SMART-LLM. <https://arxiv.org/abs/2309.10062>
- LLM-DR. <https://arxiv.org/abs/2503.00717>
- ROSClaw. <https://arxiv.org/html/2603.26997v1>
- Why Do Multi-Agent LLM Systems Fail? (MAST). <https://arxiv.org/html/2503.13657v3>
- ROS-LLM. <https://arxiv.org/abs/2406.19741>
- EmbodiedBench. <https://arxiv.org/html/2502.09560v1>

- Butter-Bench. <https://arxiv.org/html/2510.21860v1>
- PARTNR. <https://arxiv.org/abs/2411.00081>
- PARTNR (HTML). <https://arxiv.org/html/2411.00081v1>
- RoboBench. <https://arxiv.org/abs/2510.17801>
- VLABench. <https://arxiv.org/html/2412.18194v1>
- BLINK. <https://arxiv.org/abs/2404.12390>
- Evaluation of Habitat Robotics using LLMs. <https://arxiv.org/abs/2507.06157>
- DPBench: Simultaneous Coordination. <https://arxiv.org/abs/2602.13255>
- Perception Matters (uncertainty-aware segmentation). <https://arxiv.org/abs/2408.02297>
- LIBERO-Occ. <https://arxiv.org/html/2606.10862>
- SIBench. <https://arxiv.org/abs/2509.18905>
- MV-RoboBench. <https://arxiv.org/abs/2510.19400>
- Conditional Multi-Stage Failure Recovery. <https://arxiv.org/html/2507.06016>
- AgentDebug. <https://arxiv.org/abs/2509.25370>
- ROSBag MCP Server. <https://arxiv.org/abs/2511.03497>
- AgenticROS (Open Robotics Discourse).
<https://discourse.openrobotics.org/t/agenticros-connects-ros-with-openclaw-claude-codedesktop-dispatch-and-google-gemini/53699>
- LLM-as-a-Judge reliability framework. <https://arxiv.org/abs/2604.27727>
- CaP-X. <https://arxiv.org/html/2603.22435v1>
- SWE-Bench Pro. <https://arxiv.org/abs/2509.16941>
- SWE-Bench Pro (HTML). <https://arxiv.org/html/2509.16941v1>
- Why SWE-bench Verified no longer measures frontier coding capabilities (OpenAI).
<https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>
- The SWE-Bench Illusion. <https://arxiv.org/abs/2506.12286>
- SWE-MERA. <https://arxiv.org/abs/2507.11059>
- Fight Fire with Fire (ChatGPT self-verification). <https://arxiv.org/html/2405.12641v1>
- Rethinking the Value of Agent-Generated Tests. <https://arxiv.org/abs/2602.07900>
- Probing Embodied LLMs (Lockbox, higher fidelity hurts). <https://arxiv.org/html/2605.20072>

Primary-evidence sources (our own corpus):

- Per-conversation analyses, aggregate metrics, distributions, and digests:
Research/conversation-analysis/ (see INDEX.md, _analyses.json, stats.json, and _digests/).
- Raw, redacted session transcripts cited as (file, date, turn) throughout Section 4:
claude-conversations/sessions/.